```
-- ============================================================
-- INSURANCE MANAGEMENT SYSTEM - DATABASE SCHEMA
-- College DBMS Mini Project / Final Year Project
-- Database: MySQL 8.0+
-- ============================================================

CREATE DATABASE IF NOT EXISTS insurance_ms;
USE insurance_ms;

-- ============================================================
-- TABLE 1: Admin
-- Stores administrator credentials and profile data
-- ============================================================
CREATE TABLE IF NOT EXISTS admin (
    admin_id     INT AUTO_INCREMENT PRIMARY KEY,
    username     VARCHAR(50)  NOT NULL UNIQUE,
    password     VARCHAR(255) NOT NULL,          -- bcrypt hashed
    full_name    VARCHAR(100) NOT NULL,
    email        VARCHAR(100) NOT NULL UNIQUE,
    phone        VARCHAR(15),
    created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    last_login   TIMESTAMP NULL
) ENGINE=InnoDB;

-- ============================================================
-- TABLE 2: Customers
-- Stores registered customer information
-- ============================================================
CREATE TABLE IF NOT EXISTS customers (
```

```
    customer_id   INT AUTO_INCREMENT PRIMARY KEY,
    full_name     VARCHAR(100) NOT NULL,
    email         VARCHAR(100) NOT NULL UNIQUE,
    password      VARCHAR(255) NOT NULL,          -- bcrypt hashed
    phone         VARCHAR(15)  NOT NULL,
    dob           DATE         NOT NULL,
    gender        ENUM('Male','Female','Other') NOT NULL,
    address       TEXT,
    city          VARCHAR(50),
    state         VARCHAR(50),
    pincode       VARCHAR(10),
    aadhar_no     VARCHAR(20) UNIQUE,
    pan_no        VARCHAR(20) UNIQUE,
    profile_photo VARCHAR(255) DEFAULT 'default.png',
    is_active     TINYINT(1)  DEFAULT 1,
    created_at    TIMESTAMP   DEFAULT CURRENT_TIMESTAMP,
    updated_at    TIMESTAMP   DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP
) ENGINE=InnoDB;


-- ============================================================
-- TABLE 3: Policy Categories
-- Lookup table for insurance categories
-- ============================================================
CREATE TABLE IF NOT EXISTS policy_categories (
    category_id   INT AUTO_INCREMENT PRIMARY KEY,
    category_name VARCHAR(50) NOT NULL UNIQUE,
    description   TEXT,
    icon          VARCHAR(50) DEFAULT 'shield'
```

```
) ENGINE=InnoDB;


-- ============================================================
-- TABLE 4: Policies
-- Master list of insurance policies offered
-- ============================================================
CREATE TABLE IF NOT EXISTS policies (
    policy_id          INT AUTO_INCREMENT PRIMARY KEY,
    category_id        INT          NOT NULL,
    policy_name        VARCHAR(150) NOT NULL,
    policy_code        VARCHAR(20)  NOT NULL UNIQUE,
    description        TEXT,
    coverage_amount    DECIMAL(12,2) NOT NULL,
    base_premium       DECIMAL(10,2) NOT NULL,
    premium_frequency  ENUM('Monthly','Quarterly','Half-Yearly','Yearly') DEFAULT
'Yearly',
    duration_years     INT          NOT NULL DEFAULT 1,
    min_age            INT          DEFAULT 18,
    max_age            INT          DEFAULT 65,
    features           TEXT,        -- JSON string of features list
    is_active          TINYINT(1)   DEFAULT 1,
    created_at         TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
    updated_at         TIMESTAMP    DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (category_id) REFERENCES policy_categories(category_id) ON
DELETE RESTRICT
) ENGINE=InnoDB;


-- ============================================================
-- TABLE 5: Policy Purchases
```

```
-- Records which customer bought which policy
-- ============================================================
CREATE TABLE IF NOT EXISTS policy_purchases (
    purchase_id       INT AUTO_INCREMENT PRIMARY KEY,
    customer_id       INT          NOT NULL,
    policy_id         INT          NOT NULL,
    purchase_date     DATE         NOT NULL,
    start_date        DATE         NOT NULL,
    end_date          DATE         NOT NULL,
    premium_amount    DECIMAL(10,2) NOT NULL,
    sum_insured       DECIMAL(12,2) NOT NULL,
    nominee_name      VARCHAR(100),
    nominee_relation  VARCHAR(50),
    status            ENUM('Active','Expired','Cancelled','Pending') DEFAULT 'Active',
    policy_number     VARCHAR(30)  NOT NULL UNIQUE,  -- auto-generated
    created_at        TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id) ON DELETE
CASCADE,
    FOREIGN KEY (policy_id)   REFERENCES policies(policy_id)   ON DELETE
RESTRICT
) ENGINE=InnoDB;


-- ============================================================
-- TABLE 6: Payments
-- Premium payment transactions
-- ============================================================
CREATE TABLE IF NOT EXISTS payments (
    payment_id        INT AUTO_INCREMENT PRIMARY KEY,
    purchase_id       INT          NOT NULL,
    customer_id       INT          NOT NULL,
```

```
    amount           DECIMAL(10,2) NOT NULL,
    payment_date     TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
    payment_method   ENUM('Credit Card','Debit Card','Net Banking','UPI','Cash') NOT
NULL,
    transaction_id   VARCHAR(50)  NOT NULL UNIQUE,
    due_date         DATE,
    status           ENUM('Success','Failed','Pending','Refunded') DEFAULT 'Success',
    receipt_no       VARCHAR(30)  NOT NULL UNIQUE,
    notes            TEXT,
    FOREIGN KEY (purchase_id)  REFERENCES policy_purchases(purchase_id) ON
DELETE CASCADE,
    FOREIGN KEY (customer_id)  REFERENCES customers(customer_id)       ON
DELETE CASCADE
) ENGINE=InnoDB;


-- ============================================================
-- TABLE 7: Claims
-- Insurance claim requests by customers
-- ============================================================
CREATE TABLE IF NOT EXISTS claims (
    claim_id         INT AUTO_INCREMENT PRIMARY KEY,
    purchase_id      INT          NOT NULL,
    customer_id      INT          NOT NULL,
    claim_number     VARCHAR(30)  NOT NULL UNIQUE,
    incident_date    DATE         NOT NULL,
    claim_amount     DECIMAL(12,2) NOT NULL,
    reason           TEXT         NOT NULL,
    document_path    VARCHAR(255),
    status           ENUM('Pending','Under Review','Approved','Rejected','Paid') DEFAULT
'Pending',
```

```
    admin_remarks     TEXT,
    approved_amount   DECIMAL(12,2),
    filed_on          TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
    reviewed_on       TIMESTAMP    NULL,
    reviewed_by       INT          NULL,
    FOREIGN KEY (purchase_id)  REFERENCES policy_purchases(purchase_id) ON DELETE CASCADE,
    FOREIGN KEY (customer_id)  REFERENCES customers(customer_id)        ON DELETE CASCADE,
    FOREIGN KEY (reviewed_by)  REFERENCES admin(admin_id)               ON DELETE SET NULL
) ENGINE=InnoDB;


-- ============================================================
-- TABLE 8: Feedback
-- Customer feedback and ratings
-- ============================================================
CREATE TABLE IF NOT EXISTS feedback (
    feedback_id   INT AUTO_INCREMENT PRIMARY KEY,
    customer_id   INT NOT NULL,
    subject       VARCHAR(200),
    message       TEXT NOT NULL,
    rating        TINYINT DEFAULT 5 CHECK (rating BETWEEN 1 AND 5),
    status        ENUM('Open','Resolved','Closed') DEFAULT 'Open',
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id) ON DELETE CASCADE
) ENGINE=InnoDB;


-- ============================================================
```

```
-- TABLE 9: Notifications
-- System notifications for customers
-- ============================================================
CREATE TABLE IF NOT EXISTS notifications (
    notif_id      INT AUTO_INCREMENT PRIMARY KEY,
    customer_id   INT          NULL,   -- NULL means broadcast to all
    title         VARCHAR(150) NOT NULL,
    message       TEXT         NOT NULL,
    type          ENUM('Info','Warning','Success','Error') DEFAULT 'Info',
    is_read       TINYINT(1)   DEFAULT 0,
    created_at    TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- ============================================================
-- SAMPLE DATA INSERTS
-- ============================================================

-- Admin account (password: Admin@123)
INSERT INTO admin (username, password, full_name, email, phone) VALUES
('admin', '$2y$10$92IXUNpkjO0rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi', 'System Administrator', 'admin@insurems.com', '9876543210');

-- Policy Categories
INSERT INTO policy_categories (category_name, description, icon) VALUES
('Life Insurance',    'Financial security for your loved ones after you.',          'heart'),
('Health Insurance',  'Comprehensive medical coverage for you and your family.',    'activity'),
('Vehicle Insurance', 'Protect your vehicle against accidents and theft.',          'truck'),
('Travel Insurance',  'Stay protected during domestic and international travel.',   'map-pin');
```

-- Sample Policies

INSERT INTO policies (category_id, policy_name, policy_code, description, coverage_amount, base_premium, premium_frequency, duration_years, min_age, max_age, features) VALUES

(1, 'SecureLife Premier Plan', 'SLP001', 'Comprehensive life cover with accidental death benefit.', 5000000, 12000, 'Yearly', 20, 18, 60,

'["Death Benefit","Accidental Cover","Tax Benefit u/s 80C","Maturity Bonus"]'),

(1, 'FamilyShield Term Plan', 'FST002', 'Pure term life insurance for maximum coverage at low cost.', 10000000, 8500, 'Yearly', 30, 18, 55,

'["High Coverage","Low Premium","Critical Illness Rider","Tax Benefit"]'),

(2, 'HealthGuard Individual', 'HGI001', 'Individual health insurance with cashless hospitalisation.', 500000, 7200, 'Yearly', 1, 18, 65,

'["Cashless Hospitalisation","Day Care Procedures","Pre & Post Hospitalisation","Annual Health Checkup"]'),

(2, 'FamilyHealth Plus', 'FHP002', 'Floater plan covering entire family under one policy.', 1000000, 14500, 'Yearly', 1, 18, 70,

'["Family Floater","Maternity Cover","New Born Cover","Mental Health Cover"]'),

(3, 'AutoProtect Comprehensive', 'APC001', 'Complete vehicle protection including third-party liability.', 800000, 9500, 'Yearly', 1, 18, 75,

'["Own Damage Cover","Third Party Liability","Zero Depreciation","Roadside Assistance"]'),

(3, 'TwoWheeler Shield', 'TWS001', 'Affordable two-wheeler insurance with roadside assistance.', 100000, 2200, 'Yearly', 1, 18, 75,

'["Theft Protection","Accident Cover","Pillion Rider Cover","24x7 RSA"]'),

(4, 'TravelSafe Domestic', 'TSD001', 'Hassle-free domestic travel insurance.', 200000, 1500, 'Yearly', 1, 5, 80,

'["Trip Cancellation","Lost Baggage","Medical Emergency","Flight Delay"]'),

(4, 'GlobalTravel Elite', 'GTE001', 'Premium international travel insurance for global travellers.',800000, 6000, 'Yearly', 1, 5, 75,

'["Medical Evacuation","Adventure Sports","Passport Loss","Unlimited Trips"]');

-- Sample Customers (password: Test@1234)

INSERT INTO customers (full_name, email, password, phone, dob, gender, address, city, state, pincode, aadhar_no, pan_no) VALUES

('Rajesh Kumar', 'rajesh@example.com', '$2y$10$92IXUNpkjO0rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi', '9812345678', '1985-06-15', 'Male', '12, MG Road', 'Mumbai', 'Maharashtra', '400001', '1234-5678-9012', 'ABCPK1234D'),

('Priya Sharma', 'priya@example.com', '$2y$10$92IXUNpkjO0rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi', '9823456789', '1992-03-22', 'Female', '45, Park Lane', 'Delhi', 'Delhi', '110001', '2345-6789-0123', 'BCFPS2345E'),

('Amit Patel', 'amit@example.com', '$2y$10$92IXUNpkjO0rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi', '9834567890', '1990-11-08', 'Male', '7, Nehru Nagar', 'Ahmedabad','Gujarat', '380001', '3456-7890-1234', 'CDGPA3456F'),

('Sunita Singh', 'sunita@example.com', '$2y$10$92IXUNpkjO0rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi', '9845678901', '1988-07-30', 'Female', '23, Lake View', 'Bengaluru','Karnataka', '560001', '4567-8901-2345', 'DEHSS4567G'),

('Vikram Reddy', 'vikram@example.com', '$2y$10$92IXUNpkjO0rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi', '9856789012', '1995-01-17', 'Male', '89, Banjara Hills','Hyderabad','Telangana', '500034', '5678-9012-3456', 'EFIVR5678H');

-- Sample Policy Purchases

INSERT INTO policy_purchases (customer_id, policy_id, purchase_date, start_date, end_date, premium_amount, sum_insured, nominee_name, nominee_relation, status, policy_number) VALUES

(1, 1, '2024-01-10', '2024-01-10', '2044-01-09', 12000, 5000000, 'Meena Kumar', 'Spouse', 'Active', 'IMS-2024-L-00001'),

(1, 3, '2024-02-15', '2024-02-15', '2025-02-14', 7200, 500000, 'Meena Kumar', 'Spouse', 'Active', 'IMS-2024-H-00002'),

(2, 4, '2024-03-01', '2024-03-01', '2025-02-28', 14500, 1000000, 'Rahul Sharma', 'Husband', 'Active', 'IMS-2024-H-00003'),

(3, 5, '2024-04-20', '2024-04-20', '2025-04-19', 9500, 800000, 'Kavita Patel', 'Spouse', 'Active', 'IMS-2024-V-00004'),

(4, 7, '2024-05-05', '2024-05-05', '2025-05-04', 1500, 200000, 'Ravi Singh', 'Husband', 'Active', 'IMS-2024-T-00005'),

(5, 2, '2024-06-12', '2024-06-12', '2054-06-11', 8500, 10000000,'Anita Reddy', 'Spouse', 'Active', 'IMS-2024-L-00006');

-- Sample Payments

INSERT INTO payments (purchase_id, customer_id, amount, payment_method, transaction_id, due_date, status, receipt_no) VALUES

(1, 1, 12000, 'Net Banking', 'TXN20240110001', '2024-01-10', 'Success', 'RCP-2024-00001'),

(2, 1, 7200, 'UPI', 'TXN20240215002', '2024-02-15', 'Success', 'RCP-2024-00002'),

(3, 2, 14500, 'Credit Card', 'TXN20240301003', '2024-03-01', 'Success', 'RCP-2024-00003'),

(4, 3, 9500, 'Debit Card', 'TXN20240420004', '2024-04-20', 'Success', 'RCP-2024-00004'),

(5, 4, 1500, 'UPI', 'TXN20240505005', '2024-05-05', 'Success', 'RCP-2024-00005'),

(6, 5, 8500, 'Net Banking', 'TXN20240612006', '2024-06-12', 'Success', 'RCP-2024-00006');

-- Sample Claims

INSERT INTO claims (purchase_id, customer_id, claim_number, incident_date, claim_amount, reason, status, admin_remarks, approved_amount) VALUES

(2, 1, 'CLM-2024-00001', '2024-07-10', 45000, 'Hospitalisation for appendix surgery.', 'Approved', 'Documents verified. Claim approved.', 43000),

(3, 2, 'CLM-2024-00002', '2024-08-05', 120000, 'Major surgery - knee replacement.', 'Under Review', 'Documents under verification.', NULL),

(4, 3, 'CLM-2024-00003', '2024-09-15', 80000, 'Vehicle accident on highway - total damage.', 'Pending', NULL, NULL),

(1, 1, 'CLM-2024-00004', '2024-10-20', 500000, 'Critical illness benefit claim.', 'Rejected', 'Policy exclusion applies.', 0);

-- Sample Feedback

INSERT INTO feedback (customer_id, subject, message, rating) VALUES

(1, 'Excellent Service', 'The claim process was very smooth. Highly satisfied!', 5),

(2, 'Good Experience', 'Policy purchase was easy. Customer support was helpful.', 4),

(3, 'Claim Process Delayed', 'Took longer than expected for claim verification.', 3);

```
-- ============================================================
-- SQL QUERIES WITH EXPLANATIONS
-- ============================================================

-- Q1: SELECT with JOIN - Get all active policies with category names
-- Joins policies and categories to show a comprehensive policy list
SELECT
    p.policy_code,
    p.policy_name,
    pc.category_name,
    p.coverage_amount,
    p.base_premium,
    p.premium_frequency,
    p.duration_years
FROM policies p
INNER JOIN policy_categories pc ON p.category_id = pc.category_id
WHERE p.is_active = 1
ORDER BY pc.category_name, p.base_premium;

-- Q2: Aggregate - Total premium collected per month in a year
-- Uses MONTH() and SUM() to group payments by month
SELECT
    MONTH(payment_date)  AS month_num,
    MONTHNAME(payment_date) AS month_name,
    COUNT(*)             AS total_transactions,
    SUM(amount)          AS total_premium,
    AVG(amount)          AS avg_premium
FROM payments
WHERE YEAR(payment_date) = YEAR(CURDATE())
```

```
  AND status = 'Success'
GROUP BY MONTH(payment_date)
ORDER BY month_num;

-- Q3: Nested/Subquery - Customers who have more than one active policy
SELECT
    c.customer_id,
    c.full_name,
    c.email,
    (SELECT COUNT(*) FROM policy_purchases pp
     WHERE pp.customer_id = c.customer_id AND pp.status = 'Active') AS active_policies
FROM customers c
WHERE (
    SELECT COUNT(*) FROM policy_purchases pp2
    WHERE pp2.customer_id = c.customer_id AND pp2.status = 'Active'
) > 1;

-- Q4: JOIN + Aggregate - Claims summary per customer
SELECT
    c.full_name,
    c.email,
    COUNT(cl.claim_id)          AS total_claims,
    SUM(cl.claim_amount)        AS total_claimed,
    SUM(IFNULL(cl.approved_amount, 0)) AS total_approved,
    SUM(CASE WHEN cl.status = 'Approved' THEN 1 ELSE 0 END) AS approved_count,
    SUM(CASE WHEN cl.status = 'Rejected' THEN 1 ELSE 0 END) AS rejected_count
FROM customers c
LEFT JOIN claims cl ON c.customer_id = cl.customer_id
GROUP BY c.customer_id
```

```
ORDER BY total_claims DESC;


-- Q5: Complex JOIN - Full policy purchase report
SELECT
    pp.policy_number,
    c.full_name       AS customer_name,
    c.phone,
    pol.policy_name,
    pc.category_name,
    pp.start_date,
    pp.end_date,
    pp.premium_amount,
    pp.sum_insured,
    pp.status         AS policy_status
FROM policy_purchases pp
JOIN customers        c   ON pp.customer_id = c.customer_id
JOIN policies         pol ON pp.policy_id   = pol.policy_id
JOIN policy_categories pc  ON pol.category_id = pc.category_id
ORDER BY pp.purchase_date DESC;


-- Q6: Dashboard summary stats (for admin panel)
SELECT
    (SELECT COUNT(*) FROM customers WHERE is_active = 1)                    AS total_customers,
    (SELECT COUNT(*) FROM policy_purchases WHERE status = 'Active')         AS active_policies,
    (SELECT COUNT(*) FROM claims)                                           AS total_claims,
    (SELECT COUNT(*) FROM claims WHERE status = 'Approved')                 AS approved_claims,
```

```
   (SELECT IFNULL(SUM(amount),0) FROM payments WHERE status = 'Success')
AS total_revenue,
   (SELECT COUNT(*) FROM claims WHERE status = 'Pending')                AS
pending_claims;


-- Q7: UPDATE - Mark expired policies
UPDATE policy_purchases
SET status = 'Expired'
WHERE end_date < CURDATE() AND status = 'Active';


-- Q8: DELETE - Remove soft-deleted inactive customers (admin action)
-- (Using logical delete is preferred; this is for demo)
DELETE FROM customers WHERE is_active = 0 AND created_at < DATE_SUB(NOW(),
INTERVAL 1 YEAR);


-- ============================================================
-- STORED PROCEDURE: Calculate premium based on age
-- Takes policy_id and customer age, returns adjusted premium
-- ============================================================
DELIMITER //
CREATE PROCEDURE CalculatePremium(
   IN p_policy_id INT,
   IN p_age       INT,
   OUT p_premium  DECIMAL(10,2)
)
BEGIN
   DECLARE base DECIMAL(10,2);
   SELECT base_premium INTO base FROM policies WHERE policy_id = p_policy_id;
   -- Age loading: +2% per year over 40, -1% per year under 30
   IF p_age > 40 THEN
```

```
        SET p_premium = base * (1 + (p_age - 40) * 0.02);
    ELSEIF p_age < 30 THEN
        SET p_premium = base * (1 - (30 - p_age) * 0.01);
    ELSE
        SET p_premium = base;
    END IF;
END //
DELIMITER ;

-- ============================================================
-- TRIGGER: Auto-create notification when claim status changes
-- ============================================================
DELIMITER //
CREATE TRIGGER after_claim_update
AFTER UPDATE ON claims
FOR EACH ROW
BEGIN
    IF OLD.status <> NEW.status THEN
        INSERT INTO notifications (customer_id, title, message, type)
        VALUES (
            NEW.customer_id,
            CONCAT('Claim ', NEW.claim_number, ' Status Updated'),
            CONCAT('Your claim has been updated to: ', NEW.status, '. ',
IFNULL(NEW.admin_remarks, '')),
            CASE NEW.status
                WHEN 'Approved' THEN 'Success'
                WHEN 'Rejected' THEN 'Error'
                ELSE 'Info'
            END
```

```
        );
    END IF;
END //
DELIMITER ;


-- ============================================================
-- TRIGGER: Auto-generate policy number and receipt numbers
-- ============================================================
DELIMITER //
CREATE TRIGGER before_payment_insert
BEFORE INSERT ON payments
FOR EACH ROW
BEGIN
    SET NEW.receipt_no = CONCAT('RCP-', YEAR(NOW()), '-',
LPAD(FLOOR(RAND()*90000+10000), 5, '0'));
END //
DELIMITER ;
```